CS3002 - Information Security

Assignment 01

Roll Number : 23K-0594

Name : Syeda Maham Mohsin

Task 1 - Frequency Analysis

Objective

The objective of this task is to decrypt a ciphertext encrypted using a monoalphabetic substitution cipher. Frequency analysis is used to identify the substitution mapping by analyzing the frequency of individual letters, bigrams, and trigrams in the ciphertext. **Methodology**

First, the provided freq.py program was used to analyze the ciphertext and calculate the frequencies of 1-grams, 2-grams, and 3-grams. The most frequent ciphertext characters and patterns were then compared with common patterns in the English language.

The letter n was initially considered to represent e because it was the most frequent character in the ciphertext. Further analysis of the 3-gram frequencies showed that ytn occurred most frequently. This was identified as a likely representation of the common English pattern the, resulting in the initial mappings: **y → t, t → h, n → e**

These mappings were then used to identify additional letters by examining partially decrypted words and their context. The process was repeated until the substitution mapping was recovered.

Results

The frequency analysis successfully revealed the substitution mapping and allowed the ciphertext to be decrypted into readable English plaintext. The recovered plaintext was saved in plaintext.txt.

The following evidence is included:

- Screenshot of the freq.py frequency analysis output.
- Screenshot showing the decryption/substitution process.
- Screenshot of the recovered plaintext.txt.

```
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files> python freq.py
1-gram (top 20):  2-gram (top 20):       3-gram (top 20):
n: 488            yt: 115                ytn: 78
y: 373            tn: 89                 vup: 30
v: 348            mu: 74                 mur: 20
x: 291            nh: 58                 ynh: 18
u: 280            vh: 57                 xzy: 16
q: 276            hn: 57                 mxu: 14
m: 264            vu: 56                 gnq: 14
h: 235            nq: 53                 ytv: 13
t: 183            xu: 52                 nqy: 13
i: 166            up: 46                 vii: 13
p: 156            xh: 45                 bxh: 13
a: 116            yn: 44                 lvq: 12
c: 104            np: 44                 nuy: 12
z: 95             vy: 44                 vyn: 12
l: 90             nu: 42                 uvy: 11
g: 83             qy: 39                 lmu: 11
b: 83             vq: 33                 nvh: 11
r: 82             vi: 32                 cmu: 11
e: 76             gn: 32                 tmq: 10
d: 59             ay: 31                 vhn: 10
```

```
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files> notepad decrypt.py
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files> cat decrypt.py
cipher_alphabet = "abcdefghijklmnopqrstuvwxyz"
plain_alphabet = "cfmypvbrlqxwiejdsgkhnazotu"

with open("ciphertext.txt", "r") as f:
    ciphertext = f.read()

plaintext = ""

for char in ciphertext:
    if char in cipher_alphabet:
        index = cipher_alphabet.index(char)
        plaintext += plain_alphabet[index]
    else:
        plaintext += char

with open("plaintext.txt", "w") as f:
    f.write(plaintext)

print("Decryption completed. plaintext.txt created.")
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files> python decrypt.py
Decryption completed. plaintext.txt created.
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files> Get-Content plaintext.txt -To
the oscars turn  on sunday which seems about right after this long strange
awards trip the bagger feels like a nonagenarian too

the awards race was bookended by the demise of harvey weinstein at its outset
and the apparent implosion of his film company at the end and it was shaped by
the emergence of metoo times up blackgown politics armcandy activism and
a national conversation as brief and mad as a fever dream about whether there
ought to be a president winfrey the season didnt just seem extra long it was
extra long because the oscars were moved to the first weekend in march to
avoid conflicting with the closing ceremony of the winter olympics thanks
```

Conclusion

Frequency analysis was successfully used to break the monoalphabetic substitution cipher. The experiment demonstrated that statistical properties of English, particularly letter, bigram, and trigram frequencies, can be used to recover the plaintext without directly knowing the encryption key.

Task 2 — Encryption Using OpenSSL

Objective

The objective of this task is to use OpenSSL to perform symmetric-key encryption and decryption using different encryption algorithms and modes. Three different configurations were tested and the decrypted files were compared with the original plaintext to verify successful encryption and decryption.

Methodology

A plaintext file named message.txt was created containing a short secret message. OpenSSL was then used to encrypt and decrypt the file using the following configurations:

Encryption	Key Size	IV
AES-128-CBC	128-bit	Used
AES-128-ECB	128-bit	Not used
AES-256-CBC	256-bit	Used

For AES-128-CBC, the following 128-bit hexadecimal key and 128-bit IV were used:

Key: 00112233445566778899aabbccddeeff
IV: 0102030405060708090a0b0c0d0e0f10

The -K option was used to provide the encryption key, while the -iv option was used to specify the initialization vector for CBC mode. The same key and IV were supplied during decryption to correctly recover the original plaintext.

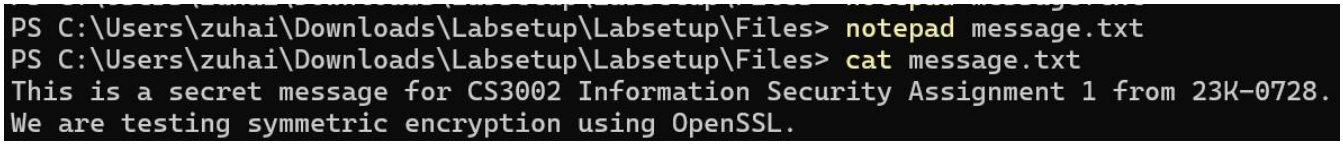
Results

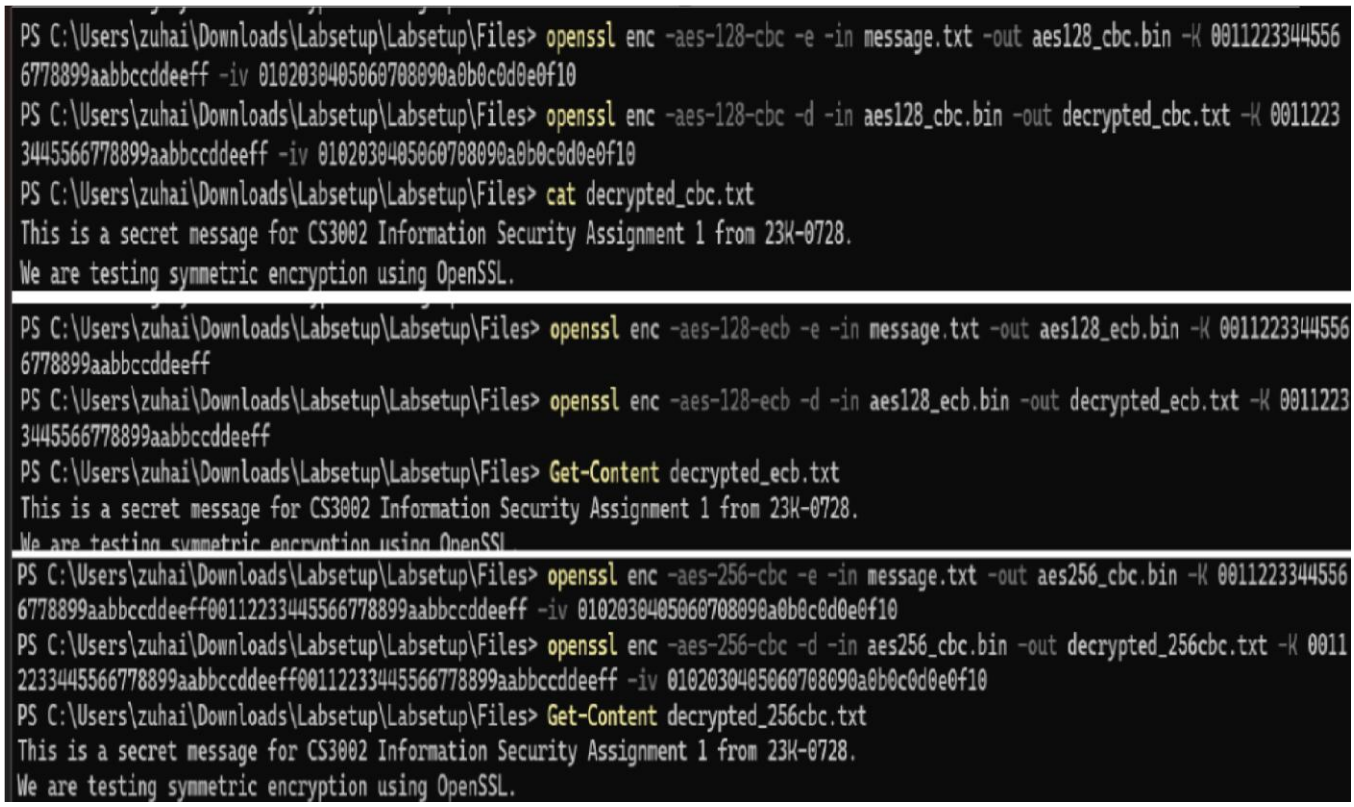
The plaintext file was successfully encrypted using all three configurations. Each resulting ciphertext file was then decrypted using the corresponding key and IV where required.

The decrypted output matched the original plaintext, confirming that the encryption and decryption operations were successful.

The following screenshots are included as evidence:

- Screenshot of the original plaintext file.
- Screenshot of AES-128-CBC encryption and decryption.
- Screenshot of AES-128-ECB encryption and decryption.
- Screenshot of AES-256-CBC encryption and decryption.





Observations

AES-128-CBC and AES-256-CBC require an initialization vector, while ECB mode does not use an IV. The key size also differs between AES-128 and AES-256. Despite these differences, all three configurations successfully recovered the original plaintext after decryption.

Conclusion

The experiment demonstrated the use of OpenSSL for symmetric-key encryption and decryption with different AES configurations. It also demonstrated the role of the encryption key and initialization vector in CBC mode and showed that the correct key and IV are required to successfully decrypt the ciphertext.

Task 3: Encryption Mode – ECB vs. CBC

Objective

The objective of this task is to understand the difference between the **Electronic Codebook (ECB)** and **Cipher Block Chaining (CBC)** modes of operation. The experiment demonstrates how ECB can reveal patterns present in the original image, while CBC provides better protection against pattern leakage.

Methodology

A BMP image was used for this experiment. Since a BMP file contains a header containing information about the image format, dimensions, and other metadata, the first **54 bytes** of the BMP file were separated from the remaining image data.

The image data was then encrypted separately using AES-128 in two different modes:

1. **AES-128-ECB**
2. **AES-128-CBC**

The following key was used:

00112233445566778899aabbccddeeff

For CBC mode, the following IV was used:

010203040506070809 0a0b0c0d0e0f10

After encryption, the original 54-byte BMP header was combined with the encrypted image data to create two new BMP files:

- pic_ecb.bmp
- pic_cbc.bmp

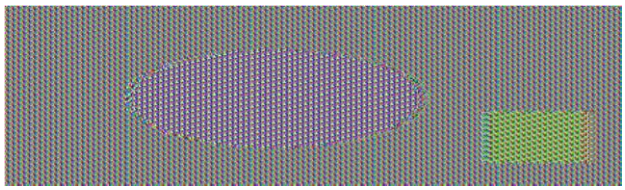
Original Image

The original BMP image was first opened to establish a reference for comparison.



ECB Encrypted Image

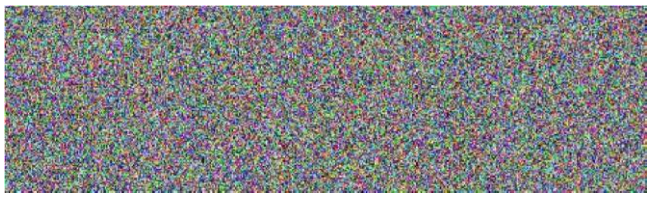
The image data was encrypted using AES-128 in ECB mode. In ECB mode, each plaintext block is encrypted independently using the same key. Therefore, identical plaintext blocks produce identical ciphertext blocks.



The encrypted image may still contain recognizable patterns from the original image. This occurs because repeated patterns in the image result in repeated plaintext blocks, which are encrypted into identical ciphertext blocks.

CBC Encrypted Image

The same image data was encrypted using AES-128 in CBC mode with the specified IV.



Unlike ECB, CBC combines each plaintext block with the previous ciphertext block before encryption. As a result, identical plaintext blocks generally produce different ciphertext blocks, making the encrypted image appear much more random.

Comparison and Observation

The experiment demonstrates an important weakness of ECB mode. Since each block is encrypted independently, repeated patterns in the original image can remain visible in the encrypted image.

CBC mode avoids this problem by chaining the encryption of consecutive blocks. Therefore, the encrypted CBC image does not preserve the visual patterns of the original image in the same way as ECB.

Mode	**Encryption of Blocks**	**Pattern Leakage**
ECB	Each block encrypted independently	High
CBC	Each block depends on previous ciphertext block	Much lower

Conclusion

This experiment demonstrates why **ECB is generally unsuitable for encrypting data containing repeated patterns**, such as images. ECB can reveal structural information even though the individual blocks are encrypted.

CBC provides better confidentiality against this type of pattern leakage because each ciphertext block depends on the previous ciphertext block. Therefore, the CBC encrypted image appears significantly more random than the ECB encrypted image.

Task 4: Padding

Objective

The objective of this task was to investigate padding requirements in different block cipher modes and observe how padding affects the size of encrypted files. AES-128CBC was also used to examine the actual padding bytes added during encryption.

Part 1: Padding in Different Encryption Modes

Four AES-128 encryption modes were tested: ECB, CBC, CFB, and OFB. A 5-byte plaintext file containing 12345 was encrypted using each mode.

The resulting ciphertext sizes were:

Encryption Mode	Plaintext Size	Ciphertext Size	Padding Required
AES-128-ECB	5 bytes	16 bytes	Yes
AES-128-CBC	5 bytes	16 bytes	Yes
AES-128-CFB	5 bytes	5 bytes	No
AES-128-OFB	5 bytes	5 bytes	No

ECB and CBC require padding because AES has a fixed block size of 16 bytes. Therefore, the 5-byte plaintext must be padded to a complete 16-byte block before encryption. CFB and OFB operate in a stream-like manner and can encrypt data that is not a complete block, so padding is not required.

```
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files> notepad f5.txt
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files> cat f5.txt
12345
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files> notepad f10.txt
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files> cat f10.txt
1234567890
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files> notepad f15.txt
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files> cat f15.txt
1234567890123456
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files> Get-Item f5.txt, f10.txt, f15.txt | Select-Object Name, Length

Name    Length
----    ------
f5.txt       5
f10.txt     10
f15.txt     16
```

Figure 4.1: Commands used and ciphertext sizes for ECB, CBC, CFB, and OFB.

Part 2: Effect of Padding on Ciphertext Size

Three plaintext files containing exactly 5, 10, and 16 bytes were created and encrypted using AES-128-CBC.

The results were:

File	Plaintext Size	Ciphertext Size
f5.txt	5 bytes	16 bytes
f10.txt	10 bytes	16 bytes
f16.txt	16 bytes	32 bytes

The 5-byte plaintext was padded with 11 bytes to reach the 16-byte AES block size. Similarly, the 10-byte plaintext was padded with 6 bytes. For the 16-byte plaintext, a complete additional 16-byte padding block was added, resulting in a ciphertext size of 32 bytes.

```
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files> openssl enc -aes-128-cbc -e -in f5.txt -out f5_cbc.bin -K 00112233445566778899aa
bbccddeeff -iv 01020304050607080090a0b0c0d0e0f10
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files>
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files> openssl enc -aes-128-cbc -e -in f10.txt -out f10_cbc.bin -K 00112233445566778899
aabbccddeeff -iv 01020304050607080090a0b0c0d0e0f10
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files>
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files> openssl enc -aes-128-cbc -e -in f16.txt -out f16_cbc.bin -K 00112233445566778899
aabbccddeeff -iv 01020304050607080090a0b0c0d0e0f10
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files> Get-Item f5_cbc.bin, f10_cbc.bin, f16_cbc.bin | Select-Object Name, Length

Name        Length
----        ------
f5_cbc.bin      16
f10_cbc.bin     16
f16_cbc.bin     32
```

Figure 4.2: Plaintext and encrypted file sizes for AES-128-CBC.

Part 3: Inspecting the Padding Bytes

The ciphertext files were decrypted using the -nopad option so that the padding bytes would not be automatically removed. The resulting files were examined using PowerShell's Format-Hex command.

For the 5-byte plaintext, the padded data was:

31 32 33 34 35 0B 0B 0B 0B 0B 0B 0B 0B 0B 0B 0B

The first five bytes represent 12345, followed by eleven 0B bytes.

For the 10-byte plaintext, the padded data was:

31 32 33 34 35 36 37 38 39 30 06 06 06 06 06 06

The first ten bytes represent 1234567890, followed by six 06 bytes.

For the 16-byte plaintext, the padded data consisted of the original 16 bytes followed by sixteen 10 bytes:

31 32 33 34 35 36 37 38 39 30 31 32 33 34 35 36 followed

by:

10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10

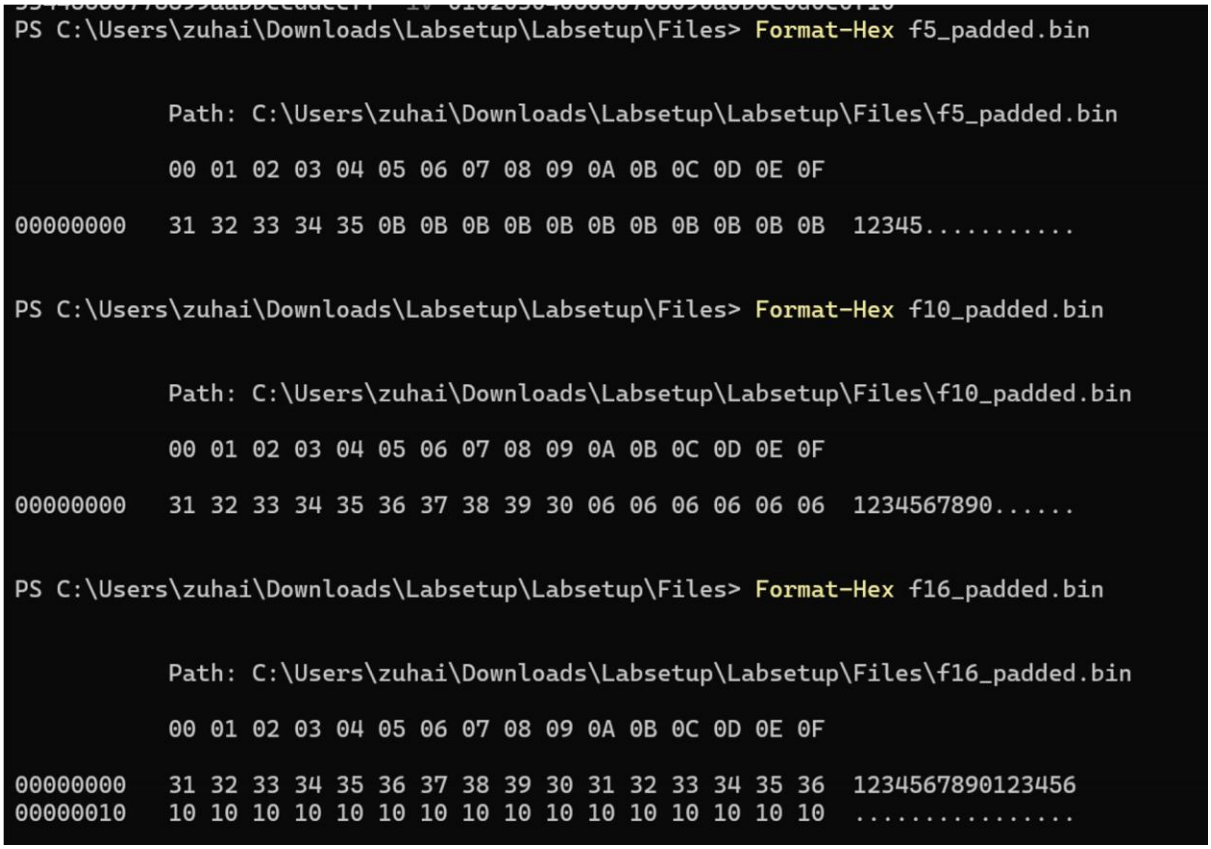


Figure 4.3: Hexadecimal representation of the padded plaintext files.

Conclusion

This task demonstrated how padding is used with block cipher modes. ECB and CBC require padding because AES processes data in fixed 16-byte blocks, whereas CFB and OFB can handle arbitrary-length plaintext without padding. The experiment also showed that when the plaintext length is already exactly one AES block, an additional full padding block is added. Using -nopad during decryption made the padding bytes visible and confirmed the padding values added to each plaintext.

Task 5: Error Propagation

Objective

The objective of this task was to investigate the effect of modifying a single bit in the ciphertext on the decrypted plaintext when using AES-128-CBC. The experiment demonstrates how errors propagate through the CBC mode during decryption.

Methodology

A plaintext file named error_test.txt containing **2522 bytes** of data was used for the experiment, satisfying the requirement of using a file larger than 1000 bytes.

The plaintext was first encrypted using AES-128-CBC with the following key and IV:

- **Key:** 00112233445566778899aabbccddeeff
- **IV:** 010203040506070809 0a0b0c0d0e0f10

The resulting ciphertext was then decrypted using the same key and IV. The original and decrypted files were compared using the fc.exe command, which produced:

FC: no differences encountered

This confirmed that the encryption and decryption process was correct.

Modification of One Ciphertext Bit

A copy of the original ciphertext was created and exactly **one bit** was modified. The byte at offset 0x10 changed from:

1A → 1B

In binary:

1A = 00011010

1B = 00011011

Therefore, only the least significant bit was changed.

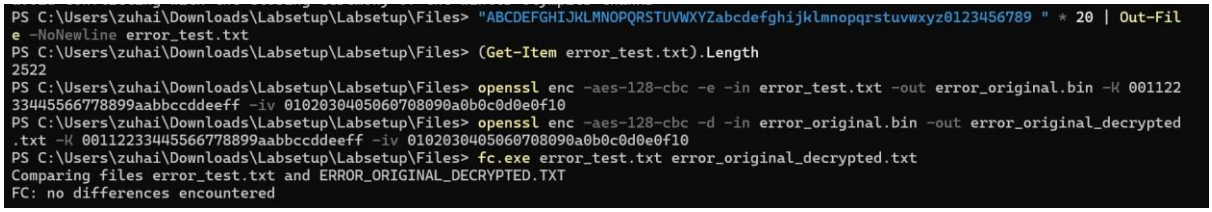


Figure 5.1: Original file size and successful encryption/decryption verification.

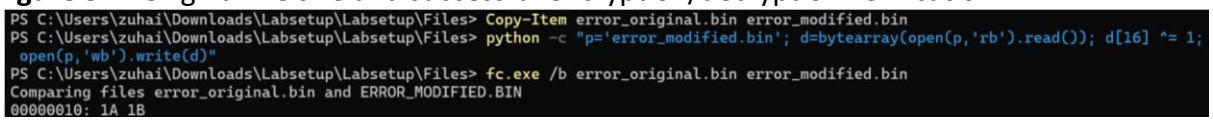


Figure 5.2: Verification that exactly one bit of the ciphertext was modified.

Decryption After the Bit Modification

The modified ciphertext was decrypted using AES-128-CBC with the same key and IV. When the resulting plaintext was compared with the original decrypted plaintext, a section of the plaintext became corrupted and contained non-printable/random characters.

The comparison showed normal plaintext before and after the corrupted section, while the affected section contained characters such as:

Pts Σ └R6Pts ▒≤■Nr╡ ╚jw▀Q

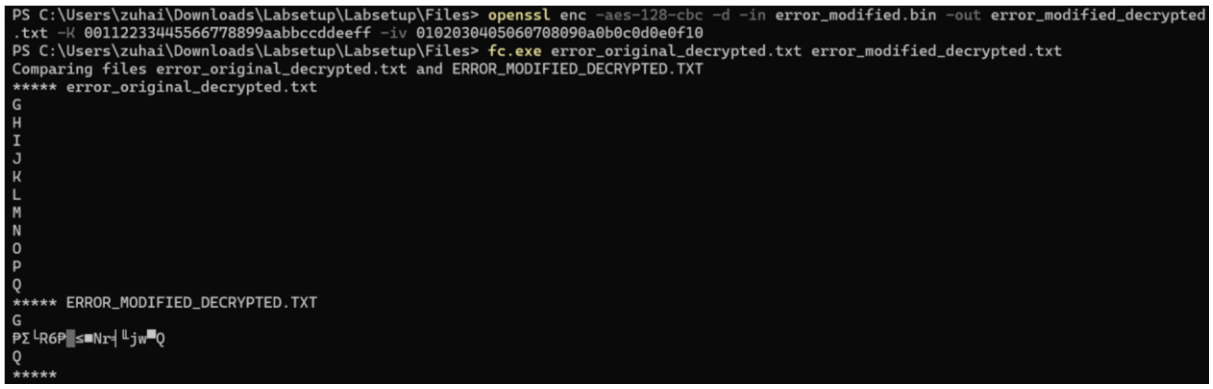


Figure 5.3: Comparison of the original and modified decrypted plaintext showing error propagation.

Observation

The experiment demonstrates the error propagation property of CBC mode. Changing one bit in a ciphertext block causes significant corruption in the corresponding decrypted plaintext block. The following plaintext block is also affected by the modified ciphertext, while subsequent blocks can recover and remain unchanged.

Thus, a small modification in the ciphertext can produce a noticeable change in the decrypted plaintext.

Conclusion

This experiment demonstrated that AES-128-CBC is sensitive to changes in ciphertext. Modifying only one bit of the ciphertext caused corruption in the decrypted plaintext, demonstrating error propagation in CBC mode. The experiment also showed that the corruption is limited to the affected block and its immediately following block, after which the plaintext can return to its original form.

Task 6: Initial Vector (IV)

6.1 IV Experiment

Objective

The objective of this task was to investigate the role of the Initial Vector (IV) in block cipher encryption. The experiment demonstrates how using different IVs affects the resulting ciphertext and why IVs should be unique when using encryption modes such as CBC.

Methodology

The same plaintext and AES-128 key were used while changing the IV. The encryption was performed using AES-128-CBC.

The following key was used:

- **Key:** 00112233445566778899aabbccddeeff

Two different IVs were used during the experiment. The plaintext was encrypted separately with each IV.

The ciphertexts produced using different IVs were compared. The same plaintext encrypted with different IVs produced different ciphertexts.

The plaintext was then encrypted twice using the **same key and same IV**. In this case, the resulting ciphertexts were identical.

```
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files> fc.exe /b iv1.bin iv1_again.bin
Comparing files iv1.bin and IV1_AGAIN.BIN
FC: no differences encountered
```

Figure 6.1: Encryption of the same plaintext using different IVs and comparison of the resulting ciphertexts.

Observation

The experiment showed that changing the IV changes the resulting ciphertext even when the plaintext and encryption key remain the same.

When the same IV was reused with the same key and plaintext, the ciphertext was identical.

This demonstrates that the IV introduces randomness into the encryption process and prevents the same plaintext from producing the same ciphertext when a new IV is used.

Conclusion

The experiment demonstrated the importance of using a unique IV during encryption. Reusing the same IV can cause identical plaintexts to produce identical ciphertexts, which may reveal information about the encrypted data.

Therefore, a new and unique IV should be used for each encryption operation.

6.2 Common Mistake: Reusing the Same IV

Objective

The objective of this task was to demonstrate the security problems caused by reusing the same IV with the same encryption key. The experiment was performed using both OFB and CFB modes.

OFB Mode

The following plaintext and ciphertext values were provided:

- **P1:** This is a known message!
- **C1:** a469b1c502c1cab966965e50425438e1bb1b5f9037a4c159
- **C2:** bf73bcd3509299d566c35b5d450337e1bb175f903fafc159

Since OFB reuses the same keystream when the same key and IV are used, the second plaintext can be recovered using:

P2 = P1 $\oplus$ C1 $\oplus$ C2

The recovered plain text was:

P2: Order: Launch a missile!

```
PS C:\Users\zuhai\Downloads\Labsetup\Labsetup\Files> python -c "p1=b'This is a known message!'; c1=bytes.fromhex('a469b1c502c1cab9669
65e50425438e1bb1b5f9037a4c159'); c2=bytes.fromhex('bf73bcd3509299d566c35b5d450337e1bb175f903fafc159'); print(bytes(a^b^c for a,b,c in
 zip(p1,c1,c2)).decode())"
Order: Launch a missile!
```

Figure 6.2: Calculation of P2 using the XOR relationship between P1, C1, and C2.

Observation

The experiment demonstrated that reusing the same IV in OFB mode causes the same keystream to be reused. If one plaintext is known, the corresponding ciphertexts can be used to recover another plaintext.

This shows that IV reuse can seriously compromise confidentiality.

CFB Mode

The same IV-reuse problem was investigated using CFB mode.

In CFB mode, the first ciphertext block depends on the IV. If the same IV is reused, the same initial keystream is generated. Therefore, if one plaintext and its corresponding ciphertext are known, information about the first block of another plaintext can be revealed.

However, after the first block, the ciphertext feedback changes the internal state. Therefore, the later blocks do not continue using the same keystream.

Observation

Only the **first 16-byte block** of the second plaintext can be revealed using the known plaintext and ciphertext pair. The remaining plaintext cannot be directly recovered using the same simple XOR relationship because the feedback mechanism changes after the first block.

Conclusion

This task demonstrated that reusing an IV is dangerous in both OFB and CFB modes. In OFB, IV reuse can allow plaintext recovery when one plaintext is known. In CFB, the first plaintext block can be exposed because the same initial keystream is reused.

Therefore, IVs should not be reused with the same key.

6.3 Predictable IV

Objective

The objective of this task was to demonstrate the security risk of using a predictable IV. An encryption oracle was used to determine whether Bob's secret message was **"Yes"** or **"No"**.

Methodology

The encryption oracle was accessed through the Docker environment. The oracle provided Bob's ciphertext, the IV used for encryption, and the next IV.

The values obtained from the oracle were:

- **Bob's ciphertext:** d2a3844a37139ce09521018e4bceb97e
- **Bob's IV:** bb28c9ca27115a8efbf4dcba1525524a
- **Next IV:** d8bb2f1928115a8efbf4dcba1525524a

The predictable nature of the IV allowed a specially crafted plaintext to be calculated.
For the candidate secret **"Yes"**, the following plaintext was generated:
3af695de020d0d0d0d0d0d0d0d0d0d0d

This crafted plaintext was submitted to the encryption oracle.

The oracle returned:

Your ciphertext:

d2a3844a37139ce09521018e4bceb97e142587a5a1a0d7a7ad9d0e81ea2235d4

The first 16 bytes of this ciphertext were:

d2a3844a37139ce09521018e4bceb97e which

exactly matched Bob's ciphertext.

```
root@7d03e178f53f:/oracle# nc 10.9.0.80 3000
Bob's secret message is either "Yes" or "No", without quotations.
Bob's ciphertex: d2a3844a37139ce09521018e4bceb97e
The IV used    : bb28c9ca27115a8efbf4dcba1525524a

Next IV        : d8bb2f1928115a8efbf4dcba1525524a
Your plaintext : 3af695de020d0d0d0d0d0d0d0d0d0d0d
Your ciphertext: d2a3844a37139ce09521018e4bceb97e142587a5a1a0d7a7ad9d0e81ea2235d4

Next IV        : c0f32a6428115a8efbf4dcba1525524a
```

Figure 6.3: Oracle output showing Bob's ciphertext, IV values, crafted plaintext, and the resulting matching ciphertext.

Observation

The first ciphertext block generated from the crafted plaintext matched Bob's ciphertext exactly. This confirmed that the guessed plaintext was the same as Bob's secret plaintext.

Therefore, Bob's secret message was determined to be: **Yes**

The attack was possible because the IV used by the encryption system was predictable.

Conclusion

This experiment demonstrated that a predictable IV can allow an attacker to perform a chosen-plaintext attack and determine secret information.

Although the encryption algorithm and key remained secure, predictable IV generation made it possible to exploit the encryption process.

Therefore, IVs should not only be unique but should also be generated in an unpredictable manner when required by the encryption mode and security design.

Task 7: Programming Using the Crypto Library

Objective

The objective of this task is to recover the AES-128-CBC encryption key using a dictionary attack and the OpenSSL Crypto library. The key is an English word shorter than 16 characters, with # characters appended to form a 128-bit (16-byte) key.

Given Information

Parameter	**Value**
Plaintext	This is a top secret.
Plaintext Length	21 characters
Encryption Algorithm	AES-128-CBC
IV	aabbccddeeff00998877665544332211
Ciphertext	764 aa26b55a4da654df6b19e4bce00f4ed05e09346fb0e762583cb7da2a c93a2

The plaintext, ciphertext, and IV were provided in the lab instructions.

Methodology

A C++ program was written using the OpenSSL Crypto library to perform a dictionary attack. The program reads words from the provided words.txt dictionary and tests each word as a possible encryption key. For each dictionary word:

1. The word is checked to ensure that its length is less than 16 characters.
2. # characters are appended until the key reaches 16 bytes.
3. AES-128-CBC encryption is performed using the given IV.
4. The generated ciphertext is compared with the given ciphertext.

5. If both ciphertexts match, the corresponding word is identified as the encryption key.

The lab specifically requires writing a program that invokes the Crypto library rather than simply using OpenSSL commands.

Generating the 128-bit Key

```
string key = word;

while (key.length() < 16)
    key += '#';
```

This appends # characters to the dictionary word until the key is exactly 16 bytes, as required for AES-128.

Testing the Candidate Key

The OpenSSL EVP interface is used to perform AES-128-CBC encryption for each candidate key.

Comparing the Ciphertext

```
if (ciphertextLen == targetCiphertext.size() &&
    memcmp(ciphertext, targetCiphertext.data(),
           targetCiphertext.size()) == 0)
{
    cout << "KEY FOUND!" << endl;
    cout << "Word: " << word << endl;
    cout << "Key : " << key << endl;
}
```

If the generated ciphertext is identical to the given ciphertext, the candidate key is considered correct.

Execution and Result

```
EVP_CIPHER_CTX *ctx = EVP_CIPHER_CTX_new();

EVP_EncryptInit_ex(ctx, EVP_aes_128_cbc(), NULL,
                   keyBytes, iv);

EVP_EncryptUpdate(ctx, ciphertext, &len,
                  plaintext, plaintextLen);

EVP_EncryptFinal_ex(ctx, ciphertext + len, &len);
EVP_CIPHER_CTX_free(ctx);
```

The program was compiled using the OpenSSL Crypto library:

g++ task7.cpp -o task7 -lcrypto

The program was then executed:

.\task7.exe

The program successfully identified the dictionary word:
KEY FOUND!

Word: Syracuse

Key : Syracuse########

```
PS C:\Fast Nuces\Semester 7\IS\Labsetup\Labsetup\Files> .\task7.exe
KEY FOUND!
Word: Syracuse
Key : Syracuse########
```

Figure 7.1: Execution of the C++ program showing the recovered encryption key. Recovered Key

The recovered English word is:

Syracuse

Since it contains 8 characters, eight # characters are appended:

Syracuse########

Therefore, the final 128-bit AES key is:

Syracuse########

Its hexadecimal representation is:

53797261637573652323232323232323

Verification

The recovered key was verified by encrypting the given plaintext using AES-128-CBC with the specified IV. The resulting ciphertext matched the ciphertext provided in the lab:

764aa26b55a4da654df6b19e4bce00f4 ed05e09346fb0e762583cb7da2ac93a2

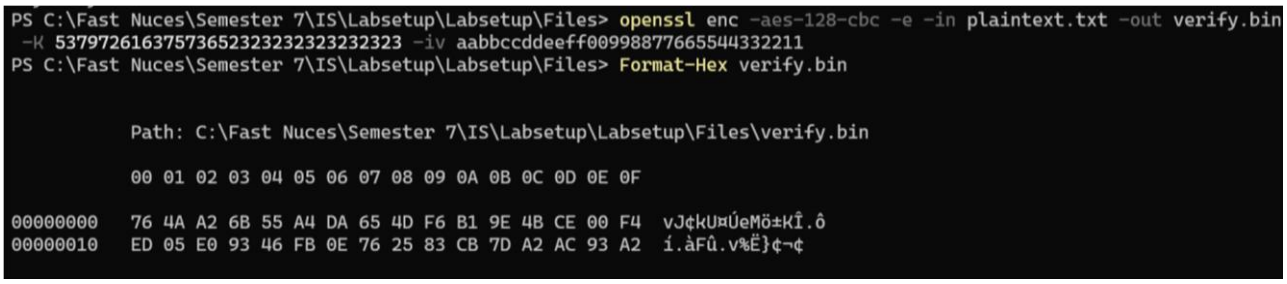


Figure 7.2: Verification of the recovered key using AES-128-CBC.

Observations

- The encryption key was successfully recovered through a dictionary attack.
- The key was not a random 128-bit value; it was derived from the English word Syracuse.
- Padding the word with # characters produced the required 16-byte AES-128 key.
- The exact ciphertext match confirms that Syracuse######## is the correct encryption key.
- Although AES-128 is designed to provide strong security, using a predictable dictionary word as the basis for the key makes the encryption vulnerable to dictionary attacks.

Conclusion

In this task, a C++ program using the OpenSSL Crypto library was developed to perform a dictionary attack against AES-128-CBC. The program successfully recovered the encryption key as **Syracuse########** by testing dictionary words and comparing their resulting ciphertexts with the given ciphertext. This experiment demonstrates the importance of using strong and unpredictable cryptographic keys rather than dictionary-based passwords.